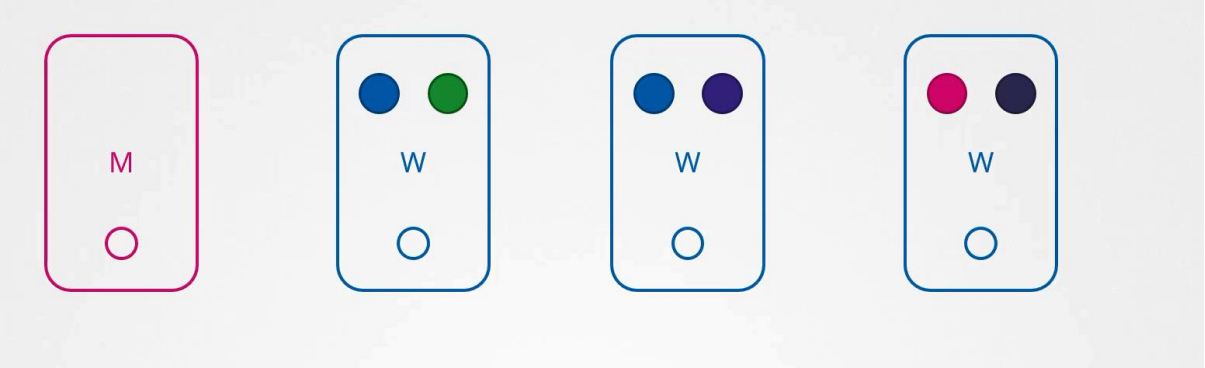
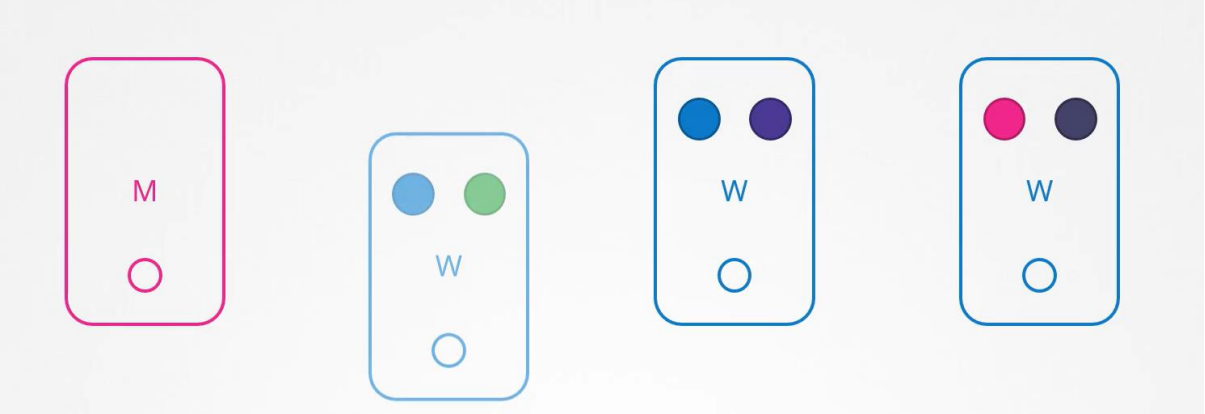
Cluster Maintenance

O/S Upgrade:



UC: What happens if the node running one blue and one green pod goes down?

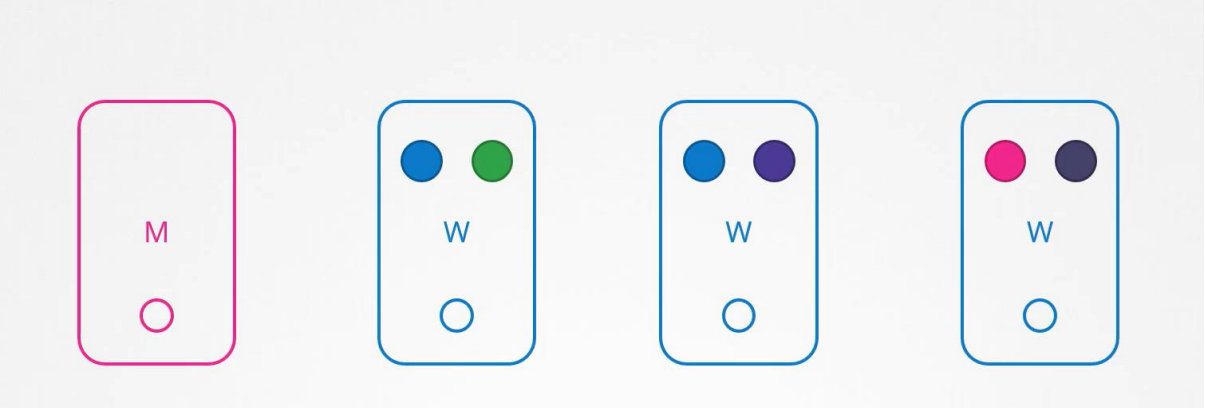


Off course pod will not available, how end user will be impacted it depends upon how you have deployed the Pod, for example since you have deployed the multiple replicas of blue pod, user accessing the blue application will not be impacted as it will be served through other blue pod that serving online, however users accessing the green pod will be impacted as there was only one green pod running the green application.

What the K8s will do in above case?

UC: If the node came online immediately

Then the kubelet's process starts and pod comeback online.

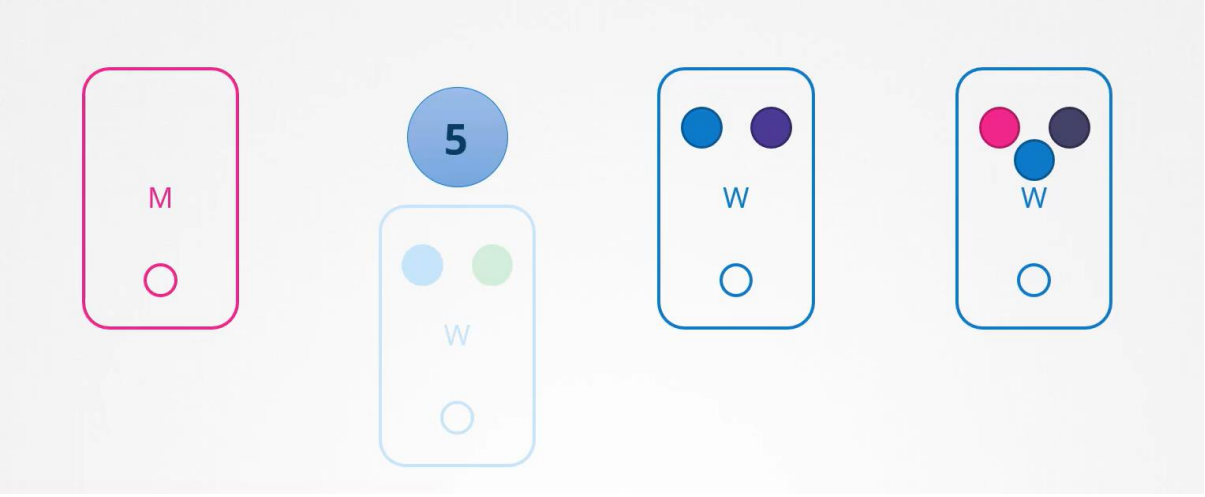


UC: If node goes down for more than 5min?

Then the Pods are terminated from that node, well K8s consider Pod as a dead. If the pods were part of your ReplicaSet then they are recreated on other nodes, the time it waits for a Pod to come back online is known as '**POD EVICTION TIME**' and is set on the kube-controller-manager with the default value is 5 min.

UC: If the node come back after Pod eviction time, it comes up with blank without any pod scheduled on it

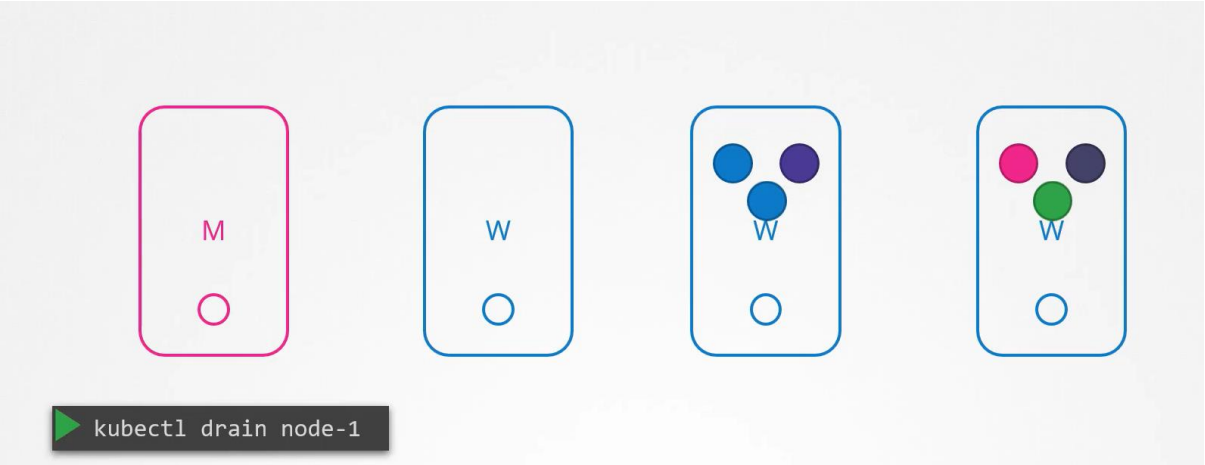
Since the **blue pod** was a part of the ReplicaSet it had new pod created on another node, however since the **green pod** was not a part of the ReplicaSet it's just gone.



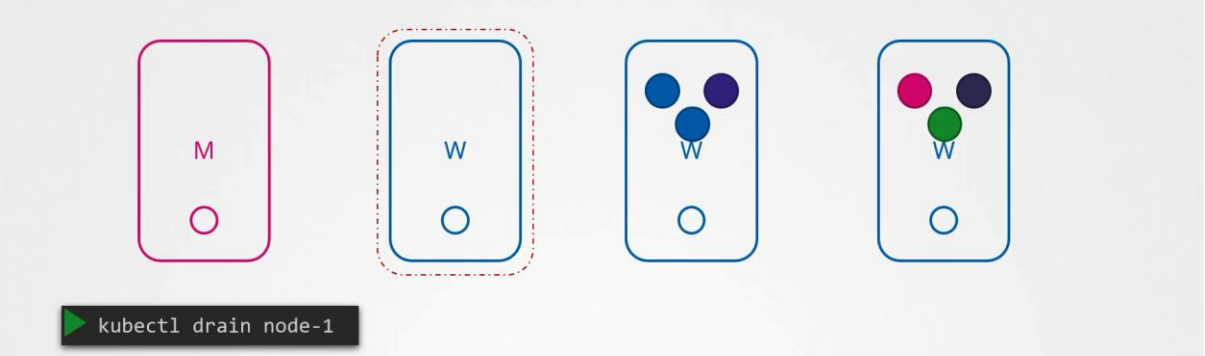
UC: You have a maintenance task to be performed on the node.

UC1: If you know that workload running on that node have other replicas and if it's OK if they come down for a short period of time and if you are sure that node will come online within 5 min you can make a quick upgrade and reboot.

UC2: If you are not sure that node will be come online in 5 min, you can purposefully drain the pods from the node. So that workload moves to another node in the cluster.

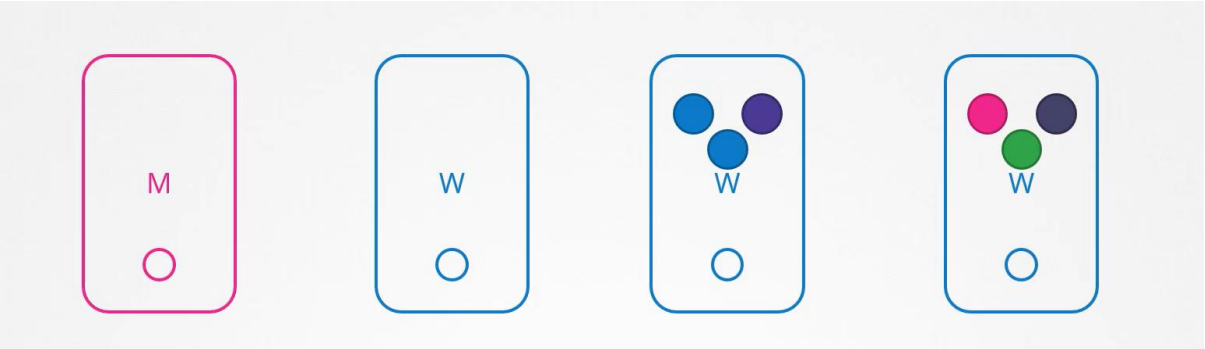


Technically they are not moved, when you drain the node, pods are gracefully terminated from the node where they are on and recreated on other, the nodes are also **cordons or marks** as un-schedulable, means no pods can be scheduled on this node. Until you specifically remove these restrictions. Now the pods are safe in another node you can reboot the first node.



When it comes back online it still un-schedulable, then you have to uncordon the node so that pod can be schedule on it again.

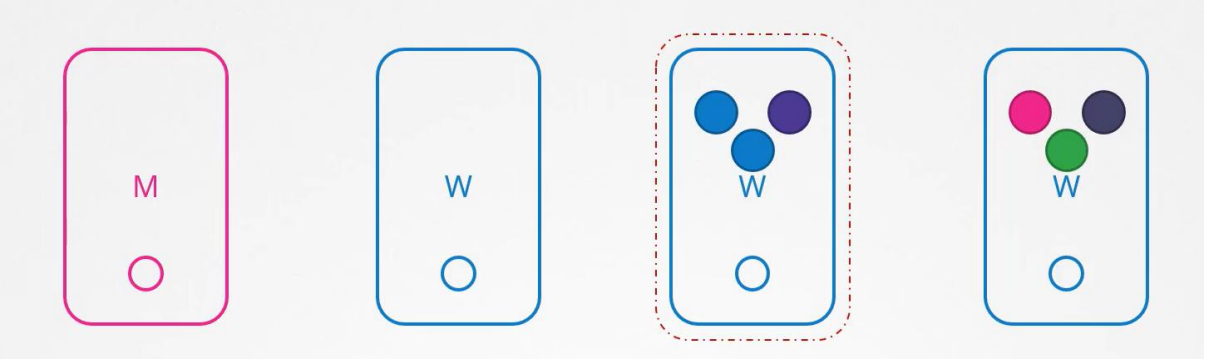
kubectl uncordon node01



The pods that were moved to another node don't automatically fall back, if any of the pods is deleted or if any new pods were created in the cluster, then they will be created in this node.

There is one more command cordon, cordon marks the node as un-schedulable, and unlike the drain it will not terminate and recreate the pod in another node it just marks the node as un-schedulable.

kubectl cordon node01



It only ensures that no pod schedule on this node.

Kubernetes Releases

When we install a K8s cluster we install a specific version of K8s.

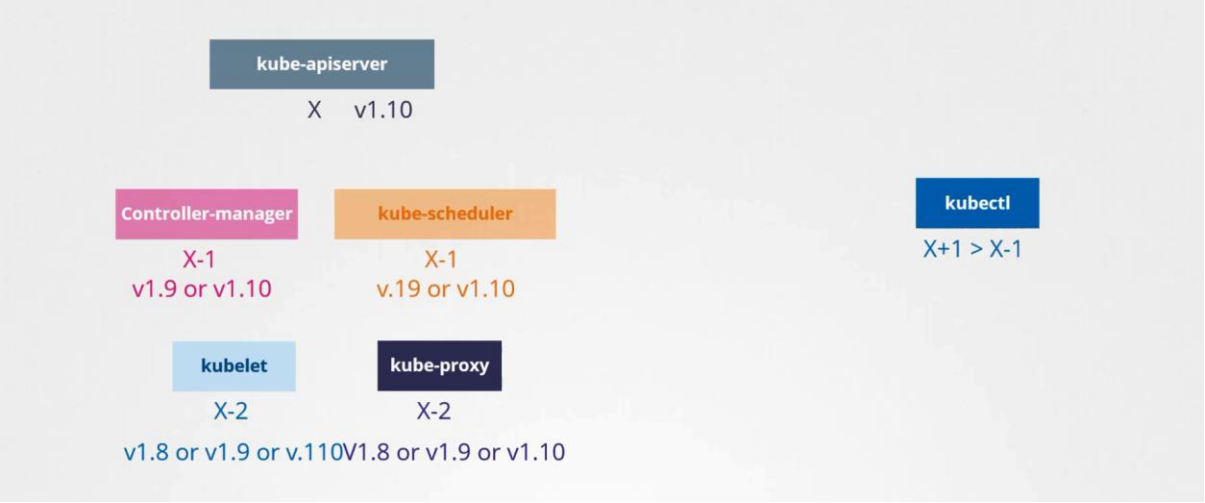
kubectl get nodes

How k8s manage these releases?

K8s version consists of three parts

Cluster Upgrade Process

kube-apiserver is the main component, none of the other component can be higher than kube-apiserver.



At any time K8s supports up to 3 recent minor versions

How to upgrade?

Upgrade one minor version at a time.

Cluster Upgrade is 2 step processes

1) Upgrade the Master
2) Upgrade the worker node

While master is upgrading control plane components (apiserver, scheduler and controller manager) goes down briefly, master goes down it does not mean worker node and applications running on worker nodes get impacted, all workload hosted on worker nodes continue to serve the user as normal, since the master components is down all management functions are down you can't access the cluster using kubectl or other k8s API. You can't deploy any new application or delete or modify existing once, the controller manager doesn't function either, if a pod will be down new pod will not be created automatically. Once the upgrade complete and master comes back online it should function normally.

k drain controlplane --ignore-daemonsets

#gracefully terminate and node marked as un-schedule-able

kubeadm upgrade plan

apt install kubeadm=1.20.0-00

apt install kubelet=1.20.0-00

kubeadm upgrade apply v1.20.0

service kubelet restart

k uncordon controlplane #marked as again schedule-able

Now it's time to upgrade the worker node, there are different strategies are available to upgrade the worker node.

Upgrade all of them at once: Then pods are down and users are not able to access the application, once the upgrade completes, nodes are back up, new pods are scheduled and the user can resume the access. This strategy requires downtime.

Upgrade one node at a time: down the first node move the workload into other nodes users are serve from there, once the first node upgraded and backup, will do the upgrade the second node and so on.

Add new nodes in the cluster: Add new nodes to the cluster with newer version, in cloud environment, move the workload on the new and remove the old node. Until you have all the nodes with latest version.

```
k cordon wnode01          #No termination just node marked as un-schedule-able
ssh wnode01
apt install kubeadm=1.20.0-00
apt install kubelet=1.20.0-00
kubeadm upgrade apply v1.20.0
service kubelet restart
exit ssh
k uncordon wnode01                #marked as again schedule-able
```